

Experiment No: 03

Machine Learning

Aim:

Implement k-NN from scratch in Python to classify Iris flowers, varying k to observe accuracy changes.

Objective:

- To understand the working of the k-Nearest Neighbors (k-NN) algorithm.
- To implement the k-NN algorithm without using sklearn's built-in classifier.
- To evaluate model performance by varying the number of neighbors (k).
- To visualize the relationship between accuracy and k.

Prerequisites:

- Software: Python 3.x with numpy, matplotlib, scikit-learn.
- Hardware: A computer with at least 4GB RAM.
- Skills: Basic knowledge of Python and supervised learning.

Theory:

The k-Nearest Neighbors (k-NN) algorithm is a supervised machine learning method used for classification and regression.

It assigns the label of a new data point based on the majority vote of its k nearest neighbors in the training dataset.

The choice of k is crucial: a small k may lead to overfitting, while a large k may smooth decision boundaries excessively.

This experiment implements k-NN from scratch and analyzes accuracy variations for different values of k using the Iris dataset.

Apparatus/Software Requirements:

- Python IDE (Jupyter Notebook/VS Code).
- Internet connection (for library installation).
- Iris dataset (available in sklearn).

Procedure:

1. Step 1: Import required libraries (numpy, matplotlib, scikit-learn).
2. Step 2: Load the Iris dataset from sklearn.datasets.
3. Step 3: Split the dataset into training and testing sets (80-20 split).
4. Step 4: Implement the k-NN algorithm from scratch using Euclidean distance.
5. Step 5: Train the model with different values of k.
6. Step 6: Predict labels for the test set and compute accuracy.
7. Step 7: Plot accuracy vs k to analyze performance trends.

Program Code:

```
import numpy as np
from collections import Counter
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt

# -----
# KNN Implementation from Scratch
# -----
class KNN:
    def __init__(self, k=3):
        self.k = k

    def fit(self, X_train, y_train):
        self.X_train = X_train
        self.y_train = y_train

    def predict(self, X_test):
        predictions = [self._predict(x) for x in X_test]
        return np.array(predictions)

    def _predict(self, x):
        distances = [np.linalg.norm(x - x_train) for x_train in self.X_train]
        k_indices = np.argsort(distances)[:self.k]
        k_nearest_labels = [self.y_train[i] for i in k_indices]
        most_common = Counter(k_nearest_labels).most_common(1)
        return most_common[0][0]

# -----
# Load Data
# -----
iris = load_iris()
X, y = iris.data, iris.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# -----SSS
# Evaluate for different k
# -----
k_values = [1, 3, 5, 7, 9, 11, 15]
```

```

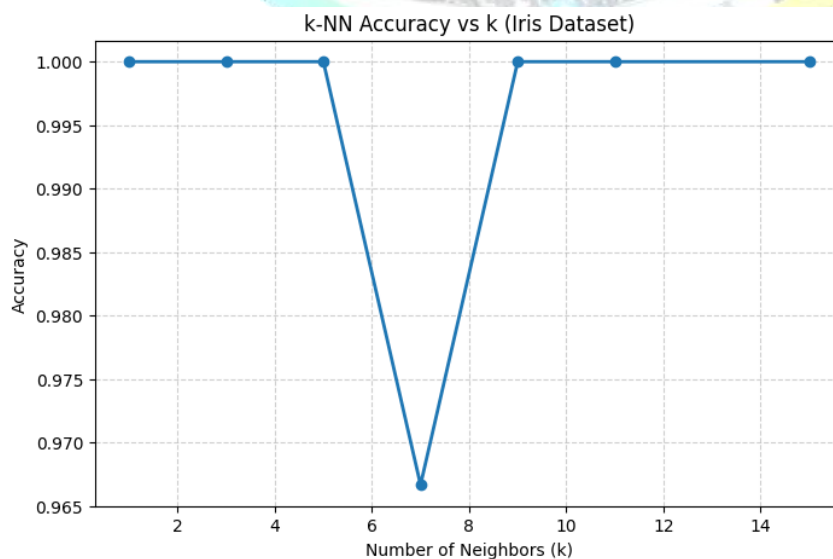
accuracies = []
for k in k_values:
    clf = KNN(k=k)
    clf.fit(X_train, y_train)
    predictions = clf.predict(X_test)
    accuracy = np.mean(predictions == y_test)
    accuracies.append(accuracy)
    print(f"k = {k}, Accuracy = {accuracy:.4f}")

# -----
# Plot Accuracy vs k
# -----
plt.figure(figsize=(8, 5))
plt.plot(k_values, accuracies, marker='o', linestyle='-', linewidth=2)
plt.title("k-NN Accuracy vs k (Iris Dataset)")
plt.xlabel("Number of Neighbors (k)")
plt.ylabel("Accuracy")
plt.grid(True, linestyle="--", alpha=0.6)
plt.show()

```

Expected Output:

- Accuracy values printed for different k values (1, 3, 5, 7, 9, 11, 15).
- k = 1, Accuracy = 1.0000
 k = 3, Accuracy = 1.0000
 k = 5, Accuracy = 1.0000
 k = 7, Accuracy = 0.9667
 k = 9, Accuracy = 1.0000
 k = 11, Accuracy = 1.0000
 k = 15, Accuracy = 1.0000
- A plot of accuracy vs k showing performance trends.



- Best accuracy obtained at an optimal value of k .

Observations:

The accuracy of the k -NN classifier varied with different values of k . Smaller k values tended to overfit, while larger k values provided smoother but sometimes less accurate predictions. An optimal k value balanced bias and variance, giving the highest accuracy.

Conclusion:

This experiment implemented the k -NN algorithm from scratch on the Iris dataset. By analyzing accuracy across multiple k values, we observed the trade-off between bias and variance and identified the optimal k for best performance.

Viva Questions:

1. What is the role of k in the k -NN algorithm?
2. Why do we use Euclidean distance in k -NN?
3. How does the choice of k affect classification performance?
4. What are the advantages and disadvantages of k -NN?
5. Why is it important to normalize data before applying k -NN?

